Centro Federal de Educação Tecnológica de Minas Gerais
Programa de Pós-Graduação em
Modelagem Matemática e Computacional

Algoritmos e Estruturas de Dados

1ª Avaliação – 33,0 pontos

ALUNO: BRASILIO ALVES FREITAS
MATRÍCULA: 20235002202

20 de abril de 2023

Questão 1:

nº de execuções	Parte do código
–	void iterativo (int n){
–	int i, j, a;
1	a = 0
$\underbrace{1+1+1+\dots+1}_{n \text{ vezes}}$	for(i = 0; i < n; i++)
$0+1+2+\dots+n-1$	for (j = 0; j < i; j++) a += i+j;

$$f(n) = 1 + (1+1+\dots+1) + (0+1+2+\dots+n-1)$$

$$f(n) = 1 + \sum_{i=1}^{n} 1 + \sum_{i=1}^{n-1} i$$

$$f(n) = 1 + n + \frac{n(n-1)}{2}$$

$$f(n) = 1 + n + \frac{n^2 - n}{2}$$

$$f(n) = 1 + \frac{n^2 + n}{2}$$

Logo a complexidade é da ordem $O(n^2)$.

Questão 2:

O mais rápido é o algoritmo por inserção. Pois a complexidade computacional é $O(n)$ para o melhor caso e $O(n^2)$ para o caso médio e pior caso, considerando o número de comparações e movimentações. Já a ordenação por seleção a complexidade computacional é $O(n^2)$, para pior caso, melhor caso e caso médio.
Na ordenação por inserção, um vetor de entradas com chaves idênticas já estará ordenado e teremos o melhor caso, onde serão feitas $(n-1)$ comparações.
Portanto o mais rápido será o algoritmo por inserção pois, para um vetor com chaves idênticas, será de complexidade $O(n)$, enquanto por seleção será $O(n^2)$.

Questão 3:

a) Heapsort, pois a complexidade é $O(n \log n)$ para qualquer caso.

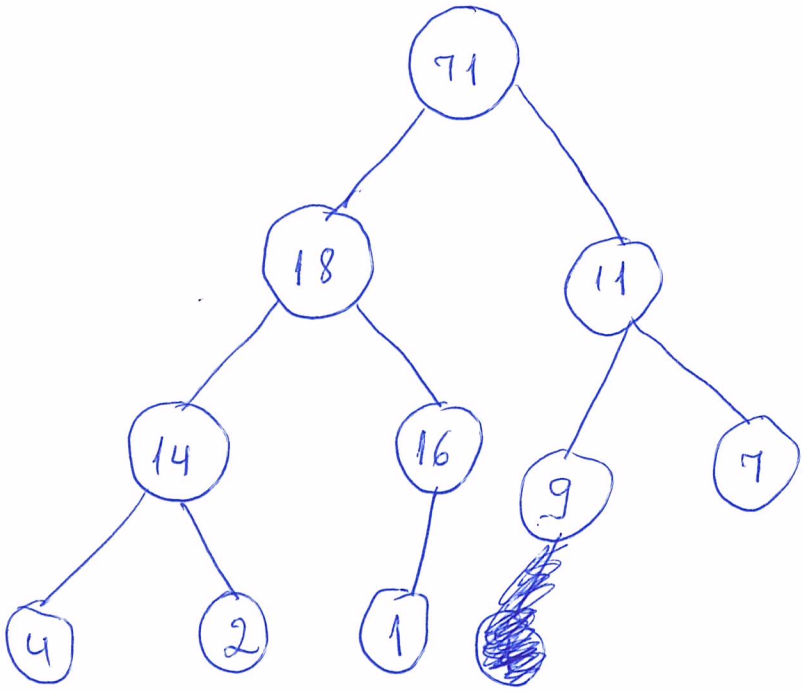
b) Por inserção, como a lista está próxima da ordem final, na prática a complexidade ficará PRÓXIMA da complexidade do melhor caso, que é $O(n)$.

Questão 4:

1	2	3	4	5	6	7	8	9	10	
4	1	7	2	16	9	11	14	18	71	Estado inicial
4	1	7	2	71	9	11	14	18	16	Esquerda = 5
4	1	7	18	71	9	11	14	2	16	Esquerda = 4
4	1	11	18	71	9	7	14	2	16	Esquerda = 3
4	71	11	18	1	9	7	14	2	16	Esquerda = 2
4	71	11	18	16	9	7	14	2	1	
71	4	11	18	16	9	7	14	2	1	Esquerda = 1
71	18	11	4	16	9	7	14	2	1	
71	18	11	14	16	9	7	4	2	1	

O heap de prioridade máxima resultante será:

71 18 11 14 16 9 7 4 2 1



Questão 5:

Parte 1

$$T(n) = T(n-1) + n - 1 \text{ , se } n > 1$$
$$T(n) = 0 \text{ , se } n = 1$$

$$\begin{array}{ll}
1 & T(n) = T(n-1) + n - 1 \\
2 & T(n-1) = T(n-2) + n - 2 \\
3 & T(n-2) = T(n-3) + n - 3 \\
\vdots & \vdots \\
x & T(n-(x-1)) = T(n-x) + n - x
\end{array}$$

$$\Rightarrow T(n) = n-1 + n-2 + n-3 + \dots + n-x + T(n-x)$$

$$T(n) = n + n + n + \dots + n - (1+2+3+\dots+x) + T(n-x)$$

$$T(n) = \sum_{i=1}^{x} n - \sum_{i=1}^{x} i + \underbrace{T(n-x)}_{=0}$$

Sendo: $\sum_{i=1}^{x} n = xn$ e $\boxed{\sum_{i=1}^{x} i = \frac{x(x+1)}{2}}$ soma de Gauss

$$T(n-x) = 0 \iff n - x = 1 \iff x = n-1$$

$$\Rightarrow T(n) = xn - \frac{x(x+1)}{2} + 0.$$

Substituindo x por $n-1$:

$$T(n) = n(n-1) - \frac{n(n-1)}{2} = \frac{2[n(n-1)] - [n(n-1)]}{2}$$

$$T(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}.$$

Assim o QUICKSORT terá ordem de complexidade $O(n^2)$.

Questão 5:

Parte 2: $\begin{cases} T(n) = 2\,T\left(\frac{n}{2}\right) + n - 1, & \text{se } n > 1 \\ T(n) = 0, & \text{se } n = 1 \end{cases}$

1. $T(n) = 2\,T\left(\frac{n}{2}\right) + n - 1$

2. $2T\left(\frac{n}{2}\right) = 2^2\,T\left(\frac{n}{2^2}\right) + n - 2$

3. $2^2T\left(\frac{n}{2^2}\right) = 2^3\,T\left(\frac{n}{2^3}\right) + n - 2^2$

$\vdots \qquad \vdots \qquad \vdots \qquad \vdots$

$x.\ 2^{x-1}T\left(\frac{n}{2^{x-1}}\right) = 2^x\,T\left(\frac{n}{2^x}\right) + n - 2^{x-1}$

$$\Rightarrow T(n) = n - 2 + n - 2^2 + \dots + n - 2^{x-1} + \underbrace{2^x\,\boxed{T\left(\frac{n}{2^x}\right)}}_{=0}$$

$$T\left(\frac{n}{2^x}\right) = 0 \iff \frac{n}{2^x} = 1 \iff \underbrace{2^x = n \iff x = \log_2 n}_{\text{Definição de logaritmo}}$$

Então: $T(n) = n + n + \dots + n - (2 + 2^2 + \dots + 2^{x-1}) + 0$

$$T(n) = \sum_{i=1}^{x} n - \sum_{i=1}^{x-1} 2^x$$

$$\boxed{\sum_{i=1}^{x-1} = \frac{1-2^x}{1-2} = \frac{1-2^x}{-1} = \frac{2^x-1}{1}}$$

$$T(n) = xn - (2^x - 1)$$

$$T(n) = xn - 2^x + 1$$

substituindo o valor de x por $\log_2 n$ e 2^x por n:

$$T(n) = n.\log_2 n - n + 1$$

Assim a ordem de complexidade do QUICKSORT será $O(n \log_2 n)$.